ACN PROJECT · TEAM 7

Frame-aware multipath QUIC with a learned bandwidth predictor

Streaming live lectures over two cellular connections without missing playback deadlines

Kabir Bajaj and Team 7 · 29 October 2026

The setting

- Students attend live classes on a **phone over mobile data**
- Many phones hold **two SIM cards**, two carriers: two internet connections at once
- Each one alone is often **slow and unsteady**: a few Mbit/s, 50–150 ms delay, short coverage holes
- **Multipath QUIC** runs one connection over both at the same time

THE QUESTION

A **scheduler** inside the transport decides, packet by packet, which connection carries the data. How should it decide when the data is **live video**?

How a live lecture travels



I-frame: full picture, ~55 KB, every 2 s P-frame: only the changes, ~8 KB, 24 per second

- A frame is usable only when **every byte** has arrived
- The player shows frame *i* a fixed **playback delay** after it was produced (we use 300 ms)
- Later than that: a **missed deadline**, the picture freezes or glitches

The problem: schedulers see bytes, not frames

What standard schedulers do

- **MinRTT**: lowest-delay path with room
- **Round-robin**: alternate paths
- **Redundant**: everything on both paths
- **ECF**: wait for the fast path if that finishes sooner

Why that hurts video

- **Frames get split**: done only when the slow half arrives
- **Big frames block small ones** queued behind them

WORKED EXAMPLE

A 45 KB I-frame on a 1.5 Mbit/s path needs **240 ms** to transmit, plus 30 ms travel: **270 ms of a 300 ms budget**, before counting the frames already queued.

With two slow paths there is no room to spare.

Research questions

1. How often do transport-only multipath schedulers make lecture frames **miss their playback deadline** on two low-bandwidth cellular paths?
2. Can a scheduler that knows each frame's **size and deadline**, driven by a **learned prediction** of each path's capacity, do better?
3. Which part of the design makes the difference? (ablations)

Closest work: MoMQ (2026), frame-aware multipath QUIC for Starlink + Wi-Fi. Two similar low-bandwidth cellular paths are not studied.

Our approach

Path observations

every 100 ms, per path: delivery rate, send rate, queueing delay, loss, cwnd, BBR estimate

→

Neural network

predicts a cautious capacity for the next 0.5 s

→

Deadline scheduler

picks the path(s) for each video frame

- Each frame on its **own QUIC stream**; the sender tells the transport its **deadline**
- Built **inside TQUIC** (Tencent's Rust QUIC library), not as a simulation

The scheduler, per frame

$$T_p = (Q_p + S) / B_p + sRTT_p / 2$$

finish time on path p · S frame size · Q_p bytes already queued · B_p predicted capacity · $sRTT_p$ round-trip time

1. If a path finishes the frame before its deadline: **whole frame on the path that finishes first**
2. Otherwise (big I-frames): **split** so both parts finish at the same moment
3. Keep each frame on its path(s), so **small frames overtake** a big one on the other path
4. A frame already past its deadline may use any path (safety net)

Why machine learning? The estimate B_p is the hard part

Simple estimates of a path's capacity fail in opposite ways:

- **BBR's estimate** and the **"best of last 3 s"** rule are often **too high** when a link fades
- The **recent delivery rate** is far **too low** on a path that carries little traffic
- Too high is what makes a frame **miss its deadline**

ON HELD-OUT CELLULAR TRACES

BBR overestimates capacity **38.2%** of the time

"Best of last 3 s" overestimates **47.1%**

Mean delivery rate is off by **72%** (median)

The learned bandwidth predictor

- **Input (44 numbers)**: for each 100 ms tick of the last second, delivery rate, send rate, queueing delay, loss ratio; plus cwnd/RTT, bytes in flight, min RTT, BBR's estimate
- **Model**: small MLP, 44 → 32 → 32 → 4, ReLU
- **Output**: the 10th, 20th, 30th and 50th **percentile** of the capacity over the next 0.5 s
- **Loss**: quantile (pinball) loss, trained with Adam in numpy
- The scheduler uses the **10th percentile**: "at least this much, 9 times out of 10"

DEPLOYED INSIDE TQUIC

Weights compiled into Rust as constants; two small matrix products per path every 100 ms.

Rust and Python predictions agree to **0.002%** (574 live predictions).

All 566 TQUIC tests pass.

Training without leaking the test data

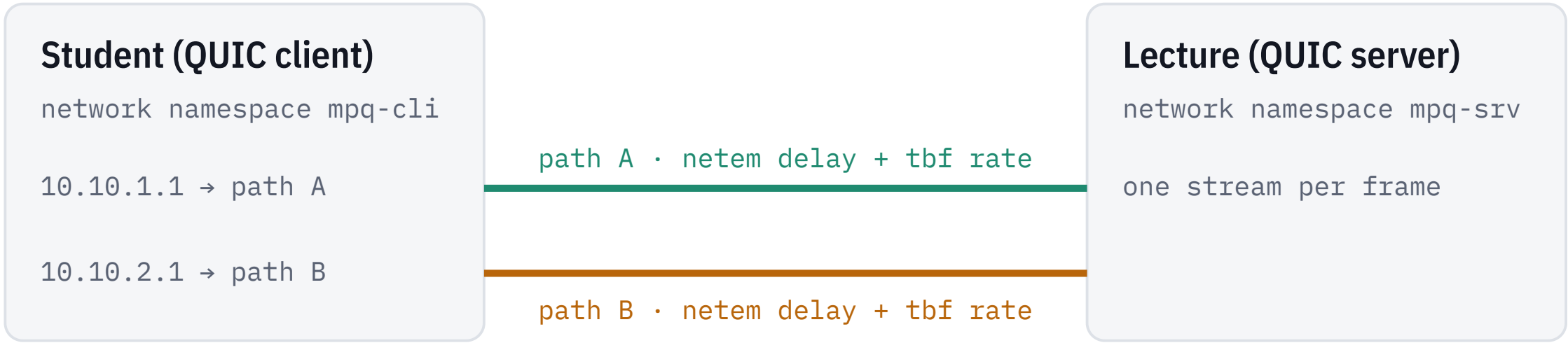
- **Training traces** (6): Verizon LTE ×2, T-Mobile LTE ×2, AT&T LTE 2016, Verizon EVDO
- **Test traces** (2, never used for training): AT&T LTE, T-Mobile 3G
- **60 emulated sessions**, 20 random conditions (traces scaled to 1–6 Mbit/s, constant rates, outages), 3 schedulers
- **Exact labels**: the emulator knows the true capacity at every moment
- Validated on **4 held-out conditions**; percentile chosen on 4 more

CALIBRATION ON HELD-OUT CONDITIONS

PERCENTILE	TARGET	MEASURED
10th	10%	12.6%
20th	20%	18.9%
30th	30%	25.5%
50th	50%	38.9%

51,210 training and 13,355 validation samples

The testbed: real code, emulated network



- Linux **network namespaces** + **veth** cables; **tc netem** adds delay, **tc tbf** sets rate and a 100 ms queue, in both directions
- A replay script changes each path's rate **every 100 ms** from real cellular traces
- Checked: 40.5 ms measured on a 40 ms path; per-path bytes match TQUIC's own count within 410 bytes

Workload and scenarios

REAL LECTURE

Harvard CopyrightX Lecture 3.2 (W. Fisher), CC BY 3.0, Wikimedia Commons

60 s clip · 1,441 frames at 24 fps · 1.57 Mbit/s · I-frames ~55 KB every 2 s

Encoded like a live stream (H.264, no B-frames, quality-capped)

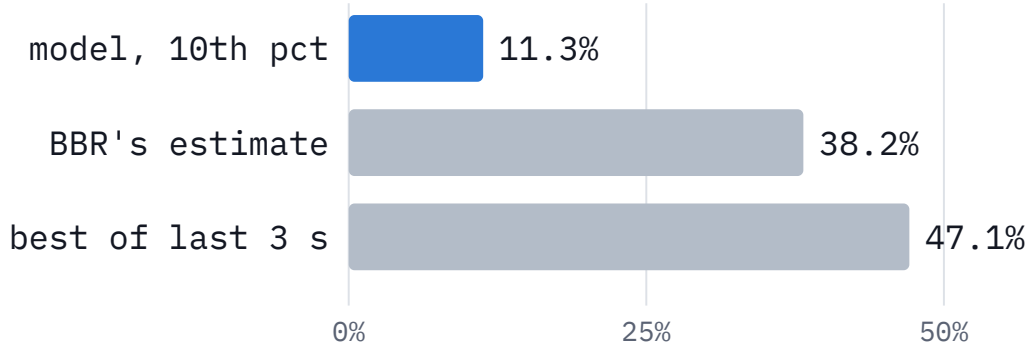
SCENARIO	PATH A	PATH B
symmetric	4 Mbit/s, 40 ms	4 Mbit/s, 40 ms
rtt_asym	4 Mbit/s, 30 ms	4 Mbit/s, 150 ms
low_bw	1.5 Mbit/s, 60 ms	1 Mbit/s, 120 ms
trace	AT&T LTE trace	T-Mobile 3G trace
outage	4 Mbit/s, stable	off 2 s every 30 s

Experiment design

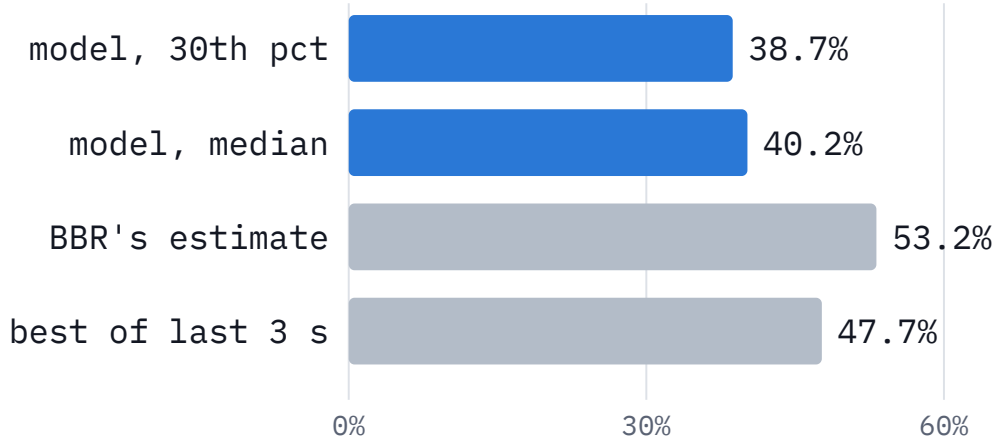
- **8 schedulers**: learned (ours), hand-tuned deadline, frame-pin, MinRTT, ECF, round-robin, redundant, single path
- **5 scenarios × 5 runs** each = **200 runs** of the 60 s lecture
- Runs with the same number see the **same network**, so schedulers are compared **run by run**
- Network rebuilt before every run; all schedulers from **one frozen build**
- Metrics: deadline-miss rate at 150 / 300 / 500 ms, I-frame misses, stall time, frame completion time, bytes per path

Result 1: the predictor is the most reliable estimate

How often each estimate is too high (trace)



Median error of the estimate (trace)



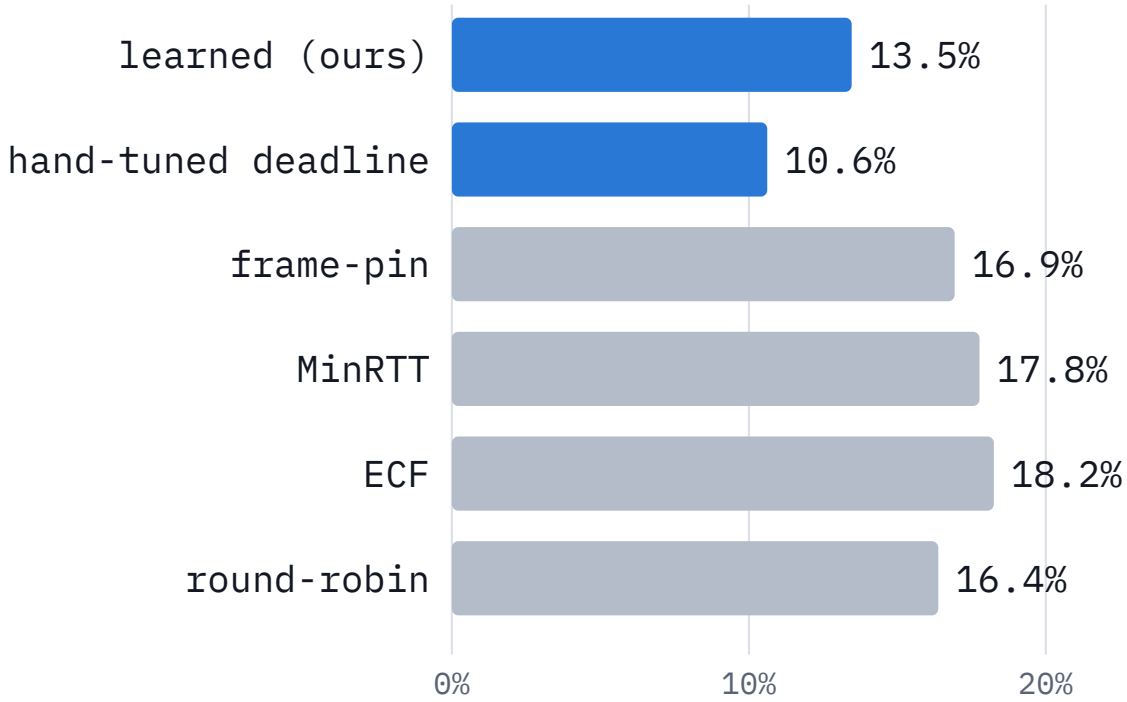
31,912 samples on traces the model never saw. Its cautious estimate overestimates a third as often as BBR's, and its estimates are closer to the truth.

Result 2: deadline misses (300 ms playback delay)

SCHEDULER	SYMMETRIC	RTT_ASYM	LOW_BW	TRACE	OUTAGE
learned (ours)	**0.1%**	**0.3%**	**4.0%**	**13.5%**	**0.6%**
hand-tuned deadline	0.1%	0.1%	3.5%	**10.6%**	0.3%
frame-pin	0.2%	0.1%	3.6%	16.9%	**0.3%**
MinRTT	**0.0%**	**0.0%**	4.7%	17.8%	0.5%
ECF	**0.0%**	**0.0%**	4.9%	18.2%	0.4%
round-robin	**0.0%**	**0.0%**	**3.3%**	16.4%	0.5%
redundant	0.2%	0.1%	99.7%	52.8%	0.5%
single path	0.5%	**0.0%**	99.6%	72.9%	0.3%

Medians over 5 runs. Fast links (symmetric, rtt_asym): almost no misses for anyone. low_bw: big I-frames barely fit. trace: real variability, where scheduling matters.

Result 3: real cellular traces



redundant 52.8% and single path 72.9% are off the scale.

- Deadline-aware schedulers miss **10.6%** (hand-tuned) and **13.5%** (learned), against **16.4%–18.2%** for transport-only
- Learned vs MinRTT / ECF: fewer misses in 3 and 3 of 5 runs, less stalling in 4 and 4 of 5
- Same order at other delays: 23.2%–24.6% vs 26.9%–28.2% at 150 ms; 5.1%–5.8% vs 7.6%–9.4% at 500 ms

What the ablations show

FRAME-PIN (NO DEADLINE CHECK, NO SPLITTING)

Misses **16.9%** on trace, no better than transport-only.

Keeping frames whole is not enough: the **deadline check and splitting** are what help.

LEARNED VS HAND-TUNED ESTIMATE

Hand-tuned 10.6%, learned 13.5%; learned better in 2 of 5 runs.

The better predictor has **not yet** become fewer misses: it is cautious everywhere and sends more traffic over the weaker path.

Findings along the way

- **A stall bug in TQUIC**: a path being validated with a full congestion window blocked every other path, ACKs included; whole connections crawled (15 of 1,800 frames in 90 s). Fixed in a few lines; worth reporting upstream.
- **Pinned frames and a rare BBR collapse**: a path fell to ~1 packet/s; frames pinned to it never arrived. Fixed by releasing late frames to any path.
- **Bitrate must fit the network**: at 2.13 Mbit/s the lecture overloaded low_bw and every scheduler missed ~65%; a live platform would lower the bitrate, so we used 1.57 Mbit/s.
- **Measurement pitfall**: WSL2 steps the wall clock by ~0.5 s; all timing uses the monotonic clock.

Limitations

- Emulation only: one laptop, public US cellular traces, no live mobile network
- Predictor trained on six public traces; transfer to Indian networks is untested
- One lecture clip, 60 s; five runs per scheduler and scenario
- Two bug fixes in TQUIC apply to every scheduler (all compared on the same build)

Next steps

1. **Turn better predictions into fewer misses**: less caution on steady paths, e.g. pick the percentile from the predicted spread
2. **Retrain with the learned scheduler's own traffic** (training data so far came from other schedulers)
3. More traces and longer lectures; figures for the final report

Summary

- Transport-only schedulers miss **16–18%** of lecture frames on real cellular traces
- Deadline-aware frame scheduling cuts that to **10.6%–13.5%**
- Our learned predictor overestimates capacity **11.3%** of the time vs **38.2%** for BBR
- Everything runs in real TQUIC code on an emulated network; code, data and results are reproducible from one repository

Thank you · Questions?